

COMPORTAMENTAL >>>>>>

State

DISCENTE: PEDRO HENRIQUE R. P. DA COSTA

DOCENTE: ROGÉRIO XAVIER AZAMBUJA

O que é o State?

- **Padrão de projeto Comportamental (GoF).**
- **Permite que um objeto mude seu comportamento quando seu estado interno muda.**
- **Dá a ilusão ao sistema de que o objeto mudou de classe em tempo de execução.**

Conceito

“O Estado é um padrão de projeto comportamental que permite que um objeto altere seu comportamento quando seu estado interno muda. Parece que o objeto mudou de classe.”

Fonte: Refactoring Guru

O Problema

- **Condicionais Monstruosas:** Estados controlados por blocos gigantes de if/else ou switch.
- **Deterioração do Software (ES I):** O software não sofre desgaste físico, mas se deteriora. Cada novo estado exige mexer em dezenas de ifs.
- **Custo de Mudança:** Pular para a manutenção com código assim custa até 100x mais (Ref: Curva de Falhas real).

A Solução

- **Dividir para Conquistar:** Extrair cada estado para novas classes isoladas.
- **Delegação:** O objeto principal (Contexto) para de checar ifs e apenas delega o trabalho para o estado ativo.
- **Transição:** Para mudar de estado, basta substituir o objeto ativo por outro.

Analogia

Imagine o botão físico de volume. A ação dele depende do estado do celular:

- **Estado Música: Aumenta o volume do áudio.**
- **Estado Chamada: Silencia o toque do telefone.**
- **Estado Câmera: Bate a foto.**

O botão é o mesmo, mas o celular delega a ação ao estado atual.

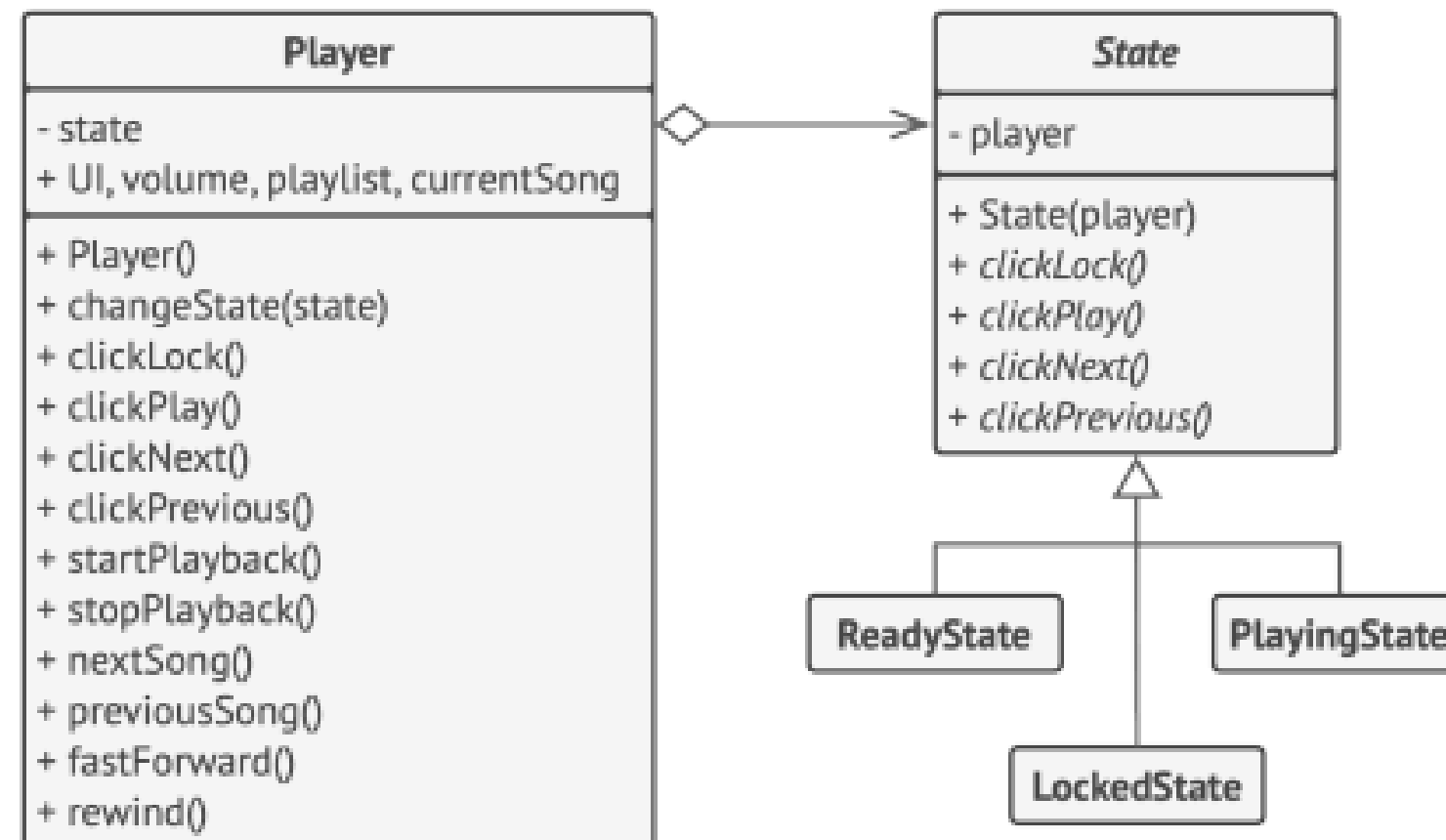
Estrutura e Atores

- **Contexto (Smartphone):** A classe que o usuário interage. Guarda o estado atual.
- **Interface State (EstadoBotao):** O contrato de métodos obrigatórios.
- **Estados Concretos:** As classes individuais (EstadoMusica, EstadoChamada, etc).

Analogia

Pseudocode

In this example, the **State** pattern lets the same controls of the media player behave differently, depending on the current playback state.



Example of changing object behavior with state objects.

Exemplo de Código

Conclusão

-  **Prós: Fim dos ifs gigantes, fácil manutenção, respeita a Responsabilidade Única.**
-  **Contras: Overengineering (excesso de arquitetura) se o sistema tiver poucos estados que raramente mudam.**

Referências

SHVETS, Alexander / Refactoring Guru. Padrão de Projeto Comportamental: State. Disponível em: <https://refactoring.guru/design-patterns/state>. Acesso em: 28 set. 2026.

**Obrigado pela
atenção!**